03 SEPTEMBER 2026

Coding Café

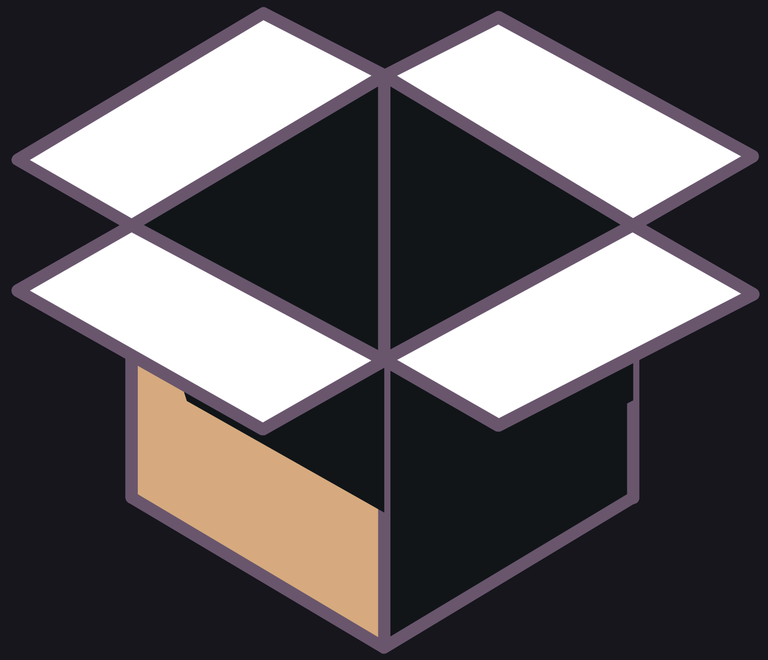
Variables, data types & operators

Outline

1. Variables
2. Intro to functions
3. Data types
4. Typecasting
5. Operators
6. Talking to the user

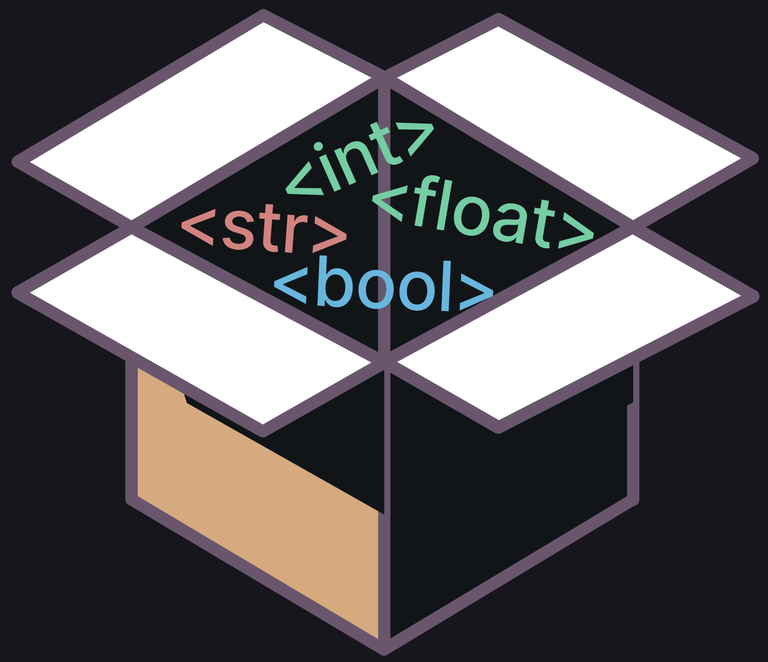
What is a Variable?

1. Containers for data



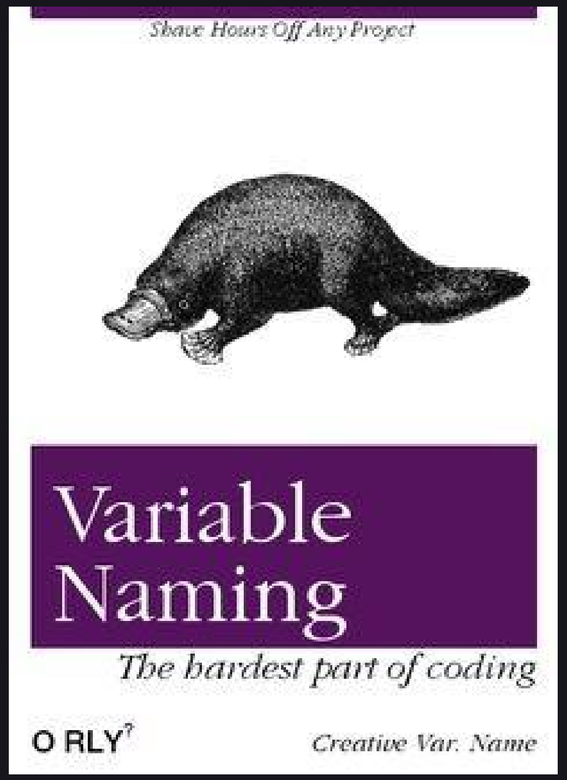
1. Containers for data
2. Type assigned at runtime

Unlike C, you don’t need to declare the data type of the value that needs to be stored in the variable.



VARIABLES

1. Containers for data
2. Type assigned at runtime
3. Should represent the semantic purpose



VARIABLES

1. Containers for data
2. Type assigned at runtime
3. Should represent the semantic purpose
4. Case sensitive

```
hundred = 100
Hundred = 100


```

VARIABLES

1. Containers for data
2. Type assigned at runtime
3. Should represent the semantic purpose
4. Case sensitive
5. Can start with:
    a. underscore ‘_’
    b. upper case letter
    c. lower case letter
6. Cannot:
    a. Start with digits
    b. Be a Python keyword *

```
1   hundred = 100
2   Hundred = 100
3   _hundred = 100
4   one100 = 100
5   1hundred = 100
```

* https://docs.python.org/3/reference/lexical_analysis.html#keywords

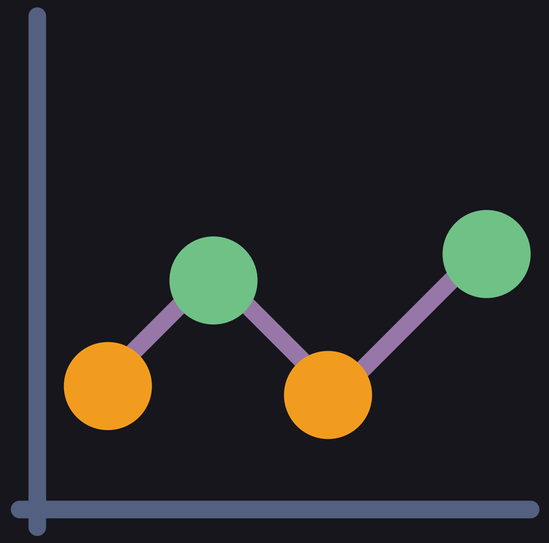
Outline

1. Variables
2. Intro to functions
3. Data types
4. Typecasting
5. Operators
6. Talking to the user

What is a Function?

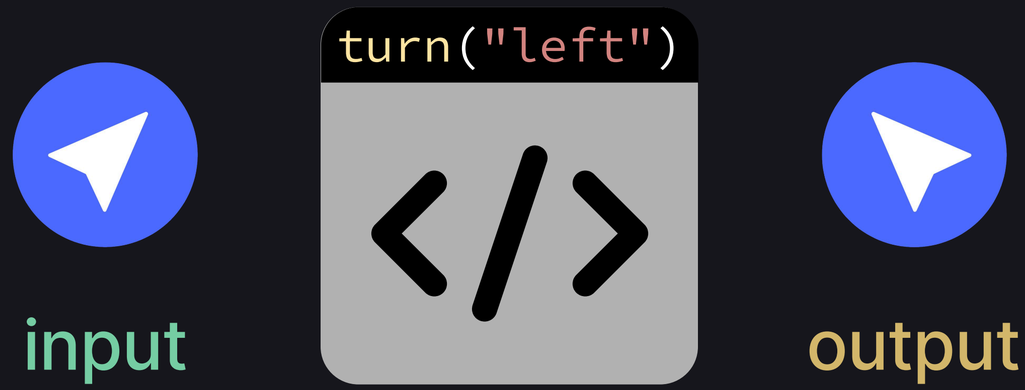
1. The mathematical function

Takes an input/a set of inputs (x) and maps them to exactly one output (y):



2. The programatic function

Takes an input state, performs an action on it or using it, and returns a new output state:



Functions can be called by passing inputs to them as well as without them:

```
turn_left()
move()
```

Without inputs

```
input("enter your name: ")
print("hello")
int("24")
```

With inputs

```
name = input("enter your name: ")
num = int("24")
```

Returning outputs to a variable

Outline

1. Variables
2. Intro to functions
3. Data types
4. Typecasting
5. Operators
6. Talking to the user

What is the Type of a Value?

DATA TYPES - PRIMITIVES

```
1   number = 10     # I am an integer
2   name = "hello" # I am a string
3   marks = 72.8    # I am a float
4   empty = True    # I am a boolean
```

DATA TYPES - PRIMITIVES

```
1   number = 10     # I am an integer
2   name = "hello" # I am a string
3   marks = 72.8    # I am a float
4   empty = True    # I am a boolean
```

All data types come with their own associated functions - This is Object Model of Python

https://docs.python.org/3/library/stdtypes.html

DATA TYPES - PRIMITIVES

```python
number = 10     # I am an integer
name = "hello" # I am a string
marks = 72.8    # I am a float
empty = True    # I am a boolean
```

All data types come with their own associated functions - This is Object Model of Python

```
a = 'hello'
a.upper()
>>> 'HELLO'
```

https://docs.python.org/3/library/stdtypes.html

DATA TYPES - PRIMITIVES

```python
number = 10     # I am an integer
name = "hello" # I am a string
marks = 72.8    # I am a float
empty = True    # I am a boolean
```

All data types come with their own associated functions - This is Object Model of Python

```
a = 'hello'
a.upper()
>>> 'HELLO'
```

```
a = 10
a.as_integer_ratio()
>>> (10, 1)
```

https://docs.python.org/3/library/stdtypes.html

How is the Type determined?

C makes you decide the type up front.

C

```c
int    number = 10;
char   name[]  = "hello";
float  marks   = 72.8;
bool   empty   = true;
```

C makes you decide the type up front.

C

```c
int    number = 10;
char   name[]  = "hello";
float  marks   = 72.8;
bool   empty   = true;
```

Python decides the type as it runs on its own.

Python

```python
number = 10
name   = "hello"
marks  = 72.8
empty  = True
```

C makes you decide the type up front.

C

```c
int    number = 10;
char   name[]  = "hello";
float  marks   = 72.8;
bool   empty   = true;
```

The declaration decides the type.

Terminology - Compile Time

Python decides the type as it runs on its own.

Python

```python
number = 10
name   = "hello"
marks  = 72.8
empty  = True
```

The value itself carries the type.

Terminology - Run Time

Outline

1. Variables
2. Intro to functions
3. Data types
4. Typecasting
5. Operators
6. Talking to the user

Sometimes you have to explicitly say what you mean.

pro_grammer@plaksha — 80 × 24

```
pro_grammer ~ $ python
>>> "6" * 3
'666'
>>> 6 * 3
18
>>>
>>> int("6") * 3
18
>>> str(6) * 3
'666'
>>> type("6")
<class 'str'>
>>> type(6)
<class 'int'>
```

`int()`, `float()`, `str()`, `bool()`

Similar characters, different data types, completely different results.

The type is not decoration. It decides what the operator actually does.

Type is associated to the behaviour of the value.

`type()` is a function shows you the data type of any value you give it.

Everything the user types is text.

pro_grammer@plaksha — 80 × 24

```
pro_grammer ~ $ python
>>> age = input("How old are you? ")
How old are you? 30
>>> age + 1
TypeError: can only concatenate str (not "int") to str

>>> age = int(input("How old are you? "))
>>> age + 1
How old are you? 30
31
```

`input()` is a function that always hands back a string.

Wrap it in `int()` or `float()` before you do any arithmetic with it.

If the user types something that is not a number, `int()` will complain and throw and error. We will learn to handle this later in the course.

Outline

1. Variables
2. Intro to functions
3. Data types
4. Typecasting
5. Operators
6. Talking to the user

OPERATIONS

Predefined format for performing operations on data

(Syntax!)

OPERATIONS

Predefined format for performing operations on data
(Syntax!)

For `int, float`

Arithmetic

- `a + b` (addition)
- `a - b` (subtraction)
- `a * b` (multiplication)
- `a // b` (truncation)
- `a / b` (division)
- `a % b` (modulo)
- `a ** b` (exponentiation)

Comparison

- `<, <=, >, >=, !=, ==`

OPERATIONS

Predefined format for performing operations on data (Syntax!)

For `int, float`

Arithmetic

- `a + b` (addition)
- `a - b` (subtraction)
- `a * b` (multiplication)
- `a // b` (truncation)
- `a / b` (division)
- `a % b` (modulo)
- `a ** b` (exponentiation)

Comparison

- `<, <=, >, >=, !=, ==`

For `strings`

Arithmetic?

- `+` does concatenation or joining
  (`'It' + ' ' + 'is' -> 'It is'`)
- `*` does repetition
  (`'6' * 3 -> '666'`)

Comparison

- `!=, ==`

OPERATIONS

Predefined format for performing operations on data (Syntax!)

For `int, float`

Arithmetic

- `a + b` (addition)
- `a - b` (subtraction)
- `a * b` (multiplication)
- `a // b` (truncation)
- `a / b` (division)
- `a % b` (modulo)
- `a ** b` (exponentiation)

Comparison

- `<, <=, >, >=, !=, ==`

For `strings`

Arithmetic?

- `+` does concatenation or joining (`'It' + ' ' + 'is' -> 'It is'`)
- `*` does repetition (`'6' * 3 -> '666'`)

Comparison

- `!=, ==`

For `bool`

Logical

- `and, or, not`

Outline

1. Variables
2. Intro to functions
3. Data types
4. Typecasting
5. Operators
6. Talking to the user

Two functions to ask and tell: `input()` and `print()`

Accepting input from the user - `input("Some prompt to the user here")`

```
>>> customer_reply = input("What would like help with today?: ")
What would like help with today?: I FOUND I FLY IN MY DAL TODAY, I WANT A REFUND
```

Providing the output - `print("The content to be printed here")`

```
>>> print("Customer states they have the following problem:\n", customer_reply)
Customer states they have the following problem:
 I FOUND I FLY IN MY DAL TODAY, I WANT A REFUND
```

OUTPUT

The `print()` function

1. Writes to the screen
2. Pass one value, or several separated by commas
3. Every call starts a new line

```
1    print("Hello, Coding Cafe")
2    >>> Hello, Coding Cafe
3
4    print("2 + 2 =", 2 + 2)
5    >>> 2 + 2 = 4
```

INPUT

The `input()` function

1. Stops and waits for the user to type
2. Whatever they type comes back as text
3. Give it a prompt, so they know what you want

```
name = input("What's your name? ")
>>> What's your name? Raghav

print("Hello,", name)
>>> Hello, Raghav
```

F-STRINGS

Putting values inside a sentence.

```
name  = "Raghav"
marks = 72.8

print("Hello, " + name)
print("Hello,", name)
print(f"Hello, {name}")

print(f"{name} scored {marks}")
print(f"{name} scored {marks:.1f}")
```

An `f` before the quote lets you drop values straight into the string with { }.

No +, no commas, no converting numbers to strings first.

`{marks:.1f}` rounds to one decimal.

03 SEPTEMBER 2026

fin